GHID TEHNIC DE INTEGRARE
PAGINA DE ADMIN (WebServices)
ÎN A SQUARED STUDIO HUB

Manual Pas-cu-Pas de Conectare, Arhitectură Securizată & Punere în Producție

Proiect Central: A Squared Studio Hub (Monorepo Fastify 5 + React 19 PWA)
Aplicație Sursă: WebServices (GitHub: `AlinStroie/WebServices.git`)
Domeniu Producție: `asquaredstudio.ro` • Rute Admin: `/admin/*` • API: `/api/admin/*`
Autor: Antigravity AI (Google DeepMind Architecture Specialist)
Destinatar: Alex / Echipa Tehnică A Squared Studio • **Status:** Ready for Implementation

Acest document constituie planul complet de inginerie software pentru transferul și integrarea modulului de administrare al site-ului oficial **asquaredstudio.ro** (găzduit în depozitul GitHub `AlinStroie/WebServices`) în interiorul platformei private **A Squared Studio Hub**.

📌 **Context din Blueprint-ul Studio Hub v2.0:** În cadrul **Fazei 5** a monorepo-ului, modulul `site-admin` a fost auditat și configurat preliminar ca slot rezervat în `deploy/apps.config.yaml` (cu `enabled: false`). Odată cu auditarea completă a depozitului `WebServices` din contul de GitHub, componentele de administrare au fost localizate și sunt gata pentru conectare conform regulilor de bază: **R3 (Modificări Minime)**, **R4 (Regresie Identică)** și **Zero Expunere Publică (Tailscale Only)**.

1. Rezumatul Executiv & Auditul Tehnic WebServices

Repo-ul `WebServices` conține stack-ul modern de producție pentru site-ul principal al

agenției, incluzând atât interfața publică, cât și panoul de administrare intern și backend-ul asociat:

Componentă	Tehnologii Folosite	Rol & Fișiere Cheie
Frontend Public & Admin	React 19, Vite, Tailwind CSS v4, Framer Motion, Recharts, Lucide Icons	`src/pages/admin/*` (9 componente complete) `src/lib/adminApi.js` (Client API dedicat)
Backend API	Node.js, Express, Helmet, CORS, Rate Limiters, HPP, Compression	`server/src/routes/admin/*` `server/src/middleware/requireAdmin.js`
Bază de Date & ORM	PostgreSQL, Prisma ORM	`server/prisma/schema.prisma` Tabele: ContactSubmission, CaseStudy, Analytics, AdminUser
Portal Clienți	React, Vite, Clerk Authentication	`portal/` (aplicație separată pentru clienți)

1.1 Inventarul Ecranelor de Administrare Existente

În folderul `src/pages/admin/` din `WebServices` sunt deja implementate următoarele funcționalități:

- **AdminDashboard (`AdminDashboard.jsx`):** Panoul central cu carduri metrice în timp real — număr contacte noi/răspunse, studii de caz publicate/draft, sesiuni analitice, rată de conversie globală și cele mai recente leaduri.
- **Gestionare Leaduri / Contacte (`AdminContacts.jsx` & `AdminContactDetails.jsx`):** CRM complet pentru formularele trimise de potențialii clienți. Permite filtrarea pe statusuri (`NEW`, `IN_PROGRESS`, `REPLIED`, `ARCHIVED`), toggle pentru bifarea semnării

contractului (*Contract Semnat*) și ștergerea/arhivarea intrărilor.

- **CMS Studii de Caz (`AdminCaseStudies.jsx` & `AdminCaseStudyEditor.jsx`):** Editor vizual pentru crearea, actualizarea și publicarea proiectelor din portofoliul A Squared (titlu, slug, client, industrie, metrici cheie, taguri tehnologice, descriere și rezultate).
- **Modul Analytics Propriu (`AdminAnalytics.jsx`):** Monitorizare fără cookie-uri de la terți a traficului pe site — vizitatori unici, afișări de pagină, canale de achiziție/referreri, rate de conversie și grafice temporale pe 7, 30 sau 90 de zile folosind Recharts.
- **Gestionare Clienți (`AdminClients.jsx`):** Managementul conturilor de clienți și asocierea proiectelor la conturile Clerk.
- **Layout & Autentificare (`AdminLayout.jsx` & `AdminLogin.jsx`):** Schelet de navigație cu bară laterală, breadcrumbs, verificare sesiune și deconectare.

2. Strategii de Arhitectură pentru Integrare

Pentru a integra acest panou de administrare în **A Squared Studio Hub**, există două modalități arhitecturale. Ambele sunt conforme cu cerințele proiectului, însă oferă beneficii diferite:

Criteriu	Opțiunea A: Nativă PWA (Recomandată)	Opțiunea B: Iframe prin Gateway Proxy
Experiență Utilizator	Fluidă, 100% nativă pe telefoanele mobile (PWA). Fără scrollbar-uri duble, fără bare exterioare.	Funcțională, dar afișată într-un cadru `<iframe>` în interiorul Hub-ului.
Design & Teme	Aliniat perfect la **ASQUARED Design System** (culori `#171b27`, `#2c2cf3`, carduri glass, font Inter).	Păstrează design-ul exact din `WebServices`.
Autentificare	**Zero login:** Hub-ul este deja securizat prin parolă + TOTP pe 6 dispozitive. Hub-ul apelează API-ul backend direct.	Necesită recunoașterea automată a antetului `X-Hub-Operator` de către Express pentru a evita login-ul redundant.
Modificări de Cod	Se creează o pagină `SiteAdminPage.tsx` în Hub și un client API `siteAdminApi.ts`. Backend-ul WebServices rămâne neschimbat.	Modificare minimă în `vite.config.js` (suport prefix `/apps/site-admin/`) și în `requireAdmin.js`.
Efort Implementare	Aproximativ 2-3 ore (copiere și	Aproximativ 30-45 minute (doar

	adaptare JSX → TSX în componentele Hub).	configurare Docker + Gateway Fastify).

Recomandarea de Arhitectură: Abordarea Hibridă Progresivă:

1. **Etapa 1 (Imediată):** Activarea **Opțiunii B (Iframe Proxy)** în `deploy/apps.config.yaml`. Acest lucru vă oferă acces funcțional complet în sub 1 oră la toate cele 9 pagini de admin existente.
2. **Etapa 2 (Optimizare PWA):** Migrarea componentelor de vizualizare (Leaduri & Analytics) ca pagini native în `hub-web` (similar cu `FacturarePage.tsx`), pentru a putea fi consultate instant de pe telefon fără reîncărcare.

3. Planul Tehnic de Implementare Pas-cu-Pas

Pasul 1: Adaptarea Backend-ului WebServices (Suport Gateway & X-Hub-Operator)

Conform **Regulii R3(c)** din Blueprint, aplicațiile încorporate trebuie să poată fi accesate fără parolă secundară dacă cererea provine prin Gateway-ul securizat al Hub-ului. Gateway-ul Hub injectează antetul `X-Hub-Operator` (ex: *ALEX*) și `X-Hub-Device`.

În fișierul `WebServices/server/src/middleware/requireAdmin.js`, adăugați verificarea antetelor Hub-ului înainte de verificarea cookie-ului JWT:

```
// Calea: WebServices/server/src/middleware/requireAdmin.js
import { prisma } from "../lib/prisma.js";
import { getAdminCookieName, verifyAdminToken } from "../lib/adminAuth.js";

export async function requireAdmin(req, res, next) {
  try {
    // 1. Verificare punte securizată Studio Hub Gateway (Regula R3-c)
    const hubOperator = req.headers["x-hub-operator"];
    const hubDevice = req.headers["x-hub-device"];

    if (hubOperator && (hubDevice || process.env.NODE_ENV !== "production")) {
```

```
      req.admin = {
        id: "hub-operator-" + hubOperator.toLowerCase(),
        email: `${hubOperator.toLowerCase()}@asquaredstudio.ro`,
        role: "SUPERADMIN",
        isHubSession: true,
      };
      return next();
    }

    // 2. Fallback la autentificarea clasică prin Cookie JWT (pentru acces direct)
    const cookieName = getAdminCookieName();
    const token = req.cookies?.[cookieName];

    if (!token) {
      return res.status(401).json({ success: false, message: "Neautorizat." });
    }

    const decoded = verifyAdminToken(token);
    const admin = await prisma.adminUser.findUnique({
      where: { id: decoded.id },
      select: { id: true, email: true, role: true, active: true },
    });

    if (!admin || !admin.active) {
      return res.status(401).json({ success: false, message: "Sesiune invalidă." });
    }

    req.admin = { id: admin.id, email: admin.email, role: admin.role };
    return next();
  } catch {
    return res.status(401).json({ success: false, message: "Sesiune expirată." });
  }
}
```

Pasul 2: Configurarea Rutelor și a Prefixului în WebServices

Gateway-ul Studio Hub redirecționează traficul la adresa `/apps/site-admin/`. Pentru ca cererile să ajungă corect la routerul de admin, în `WebServices/server/src/app.js` se adaugă montarea routerului și pe calea Hub-ului:

```
// Calea: WebServices/server/src/app.js (în jurul liniei 113)

// Montare pe ambele rute: direct (/api/admin) și prin Gateway (/apps/site-admin/api/admin)
app.use("/api/admin", adminRoutes);
app.use("/apps/site-admin/api/admin", adminRoutes);
app.use("/apps/site-admin/api/health", (req, res) => {
  res.json({ status: "ok", name: "Site Admin Engine", timestamp: new Date().toISOString() });
});
```

Pentru compilarea interfeței frontend Vite pe sub-cale (dacă se folosește modul Iframe), în `WebServices/vite.config.js` se adaugă proprietatea `base`:

```
// Calea: WebServices/vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  base: process.env.BASE_URL || '/',
  plugins: [react()],
  // ... restul configurației existente
});
```

Pasul 3: Activarea Modulului în A Squared Studio Hub

În monorepo-ul Studio Hub, deschideți fișierul `deploy/apps.config.yaml` și activați intrarea `site-admin`:

```
# Calea: A Squared Studio Hub/deploy/apps.config.yaml
apps:
  # ... facturare și lead-finder existente ...

  - id: site-admin
    name: Site Admin
```

```
    icon: globe
    order: 3
    type: web
    mount: /apps/site-admin/
    upstream: http://localhost:5000     # În dezvoltare locală
    # upstream: http://site-admin:5000  # În producție Docker Compose
    mode: iframe
    health: /api/health
    focus_capable: true
    max_body: 10mb
    backup:
      - type: postgres
        database: webservices_db
    verify_hook: integrations/site-admin/verify.sh
    enabled: true
```

Pasul 4: Pachetul de Containerizare Docker (pentru VM Oracle Cloud)

În folderul `A Squared Studio Hub/integrations/site-admin/`, creați fișierele standard de integrare:

1. `Dockerfile` Multi-Stage

```
# Calea: integrations/site-admin/Dockerfile
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY server/package*.json ./server/
RUN cd server && npm ci --only=production
COPY server ./server
COPY --from=builder /app/dist ./server/dist
WORKDIR /app/server
EXPOSE 5000
CMD ["npm", "start"]
```

2. `docker-compose.fragment.yml`

```
# Calea: integrations/site-admin/docker-compose.fragment.yml
services:
  site-admin:
    build:
      context: /opt/studio-hub/apps/site-admin
      dockerfile: Dockerfile
    container_name: studio-hub-site-admin
    restart: unless-stopped
    environment:
      - NODE_ENV=production
      - PORT=5000
      - DATABASE_URL=postgresql://${DB_USER}:${DB_PASS}@postgres:5432/webservices_db
      - CLIENT_URL=http://localhost:8080
    networks:
      - internal_net
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/api/health"]
      interval: 30s
      timeout: 5s
      retries: 3
    mem_limit: 512m
```

3. Hook de Verificare (`verify.sh`)

```
#!/usr/bin/env bash
# Calea: integrations/site-admin/verify.sh
set -e
echo "Verificare integritate Site Admin..."
curl -sf http://localhost:5000/api/health || { echo "Healthcheck eșuat!"; exit 1; }
echo "Site Admin funcționează optim."
```

4. Integrare Nativă PWA în hub-web (Opțiunea Recomandată)

Dacă doriți ca ecranele de administrare să arate și să se miște la fel de rapid ca **Facturare** și **Lead Finder**, creați fișierul de legătură API în frontend-ul Hub-ului:

```
// Calea: apps/hub-web/src/lib/siteAdminApi.ts
```

```
const BASE = '/apps/site-admin/api/admin';

async function fetchAdmin<T>(endpoint: string, options: RequestInit = {}): Promise<T> {
  const res = await fetch(`${BASE}${endpoint}`, {
    credentials: 'include',
    headers: { 'Content-Type': 'application/json', ...options.headers },
    ...options,
  });
  if (!res.ok) throw new Error(`Eroare HTTP: ${res.statusText}`);
  return res.json();
}

export const siteAdminApi = {
  getDashboard: () => fetchAdmin<any>('/dashboard'),
  getContacts: (params = {}) => fetchAdmin<any>(`/contact-submissions`),
  updateContactStatus: (id: string, status: string) =>
    fetchAdmin<any>(`/contact-submissions/${id}/status`, {
      method: 'PATCH',
      body: JSON.stringify({ status }),
    }),
  toggleContractSigned: (id: string, contractSigned: boolean) =>
    fetchAdmin<any>(`/contact-submissions/${id}/contract-signed`, {
      method: 'PATCH',
      body: JSON.stringify({ contractSigned }),
    }),
  getAnalytics: (days = 30) => fetchAdmin<any>(`/analytics/overview?days=${days}`),
  getCaseStudies: () => fetchAdmin<any>('/case-studies'),
};
```

Apoi, în `apps/hub-web/src/pages/AppViewerPage.tsx`, adăugați condiția de randare nativă:

```
// În apps/hub-web/src/pages/AppViewerPage.tsx
if (appId === 'site-admin' && app?.mode === 'native') {
  return <SiteAdminPage />;
}
```

5. Securitate, Backup GFS & Audit Log

5.1 Protecție și Izolare prin Tailscale (Zero Expunere Publică)

Unul dintre cele mai mari avantaje ale integrării este securitatea: panoul de administrare al site-ului nu mai trebuie să aibă ruta `/admin` accesibilă oricui pe internetul public. Prin Studio Hub:

- Ruta `/admin` publică poate fi dezactivată complet sau restricționată doar la IP-ul intern al VM-ului.
- Doar cei doi operatori autorizați (Alex și partenerul), conectați prin **Tailscale** și autentificați cu codul **TOTP** pe dispozitivele înregistrate, pot deschide panoul de administrare.
- Orice tentativă neautorizată este respinsă la nivelul Gateway-ului Fastify înainte de a atinge backend-ul WebServices.

5.2 Înregistrarea Acțiunilor în Audit Log (Regula R10)

Toate acțiunile critice efectuate în panoul de administrare (modificarea statusului unui lead, ștergerea unui contact, publicarea unui studiu de caz) se înregistrează automat în tabela `audit_log` din baza de date a Hub-ului prin serviciul intern `apps/hub-api/src/services/audit.ts`:

```
// Exemplu înregistrare eveniment în Hub:
recordAudit({
  operator: session.operatorTag, // ex: "ALEX"
  device: session.deviceLabel,   // ex: "iPhone 15 Pro"
  action: "SITE_ADMIN_UPDATE",
  details: "Schimbat status contact #14 în REPLIED",
});
```

5.3 Backup Criptat Zilnic (GFS: 7 zilnice, 4 săptămânale, 12 lunare)

Datele din `WebServices` (contactele primite și studiile de caz) se adaugă în fișierul `deploy/backup.targets.yaml` din Studio Hub pentru a fi criptate asimetric cu cheia `age` și urcate automat offsite via `rclone`:

```
# În deploy/backup.targets.yaml
targets:
  - id: site-admin-db
    name: Baza de Date PostgreSQL WebServices
    type: postgres_dump
    container: postgres
    database: webservices_db
    retention:
      daily: 7
      weekly: 4
      monthly: 12
```

6. Checklist de Implementare și Verificare (Regula R4)

Pentru a asigura că totul funcționează perfect fără regresii asupra site-ului public `asquaredstudio.ro`, parcurgeți următoarea listă de verificare:

Status	Etapă	Criteriu de Acceptare
[ ]	**1. Audit Cod & Git**	S-a creat ramura `hub-integration` în depozitul `WebServices` (Regula R3).
[ ]	**2. Bridge Autentificare**	Middleware-ul `requireAdmin.js` permite accesul pe baza antetului `X-Hub-Operator`.
[ ]	**3. Regresie Publică**	Site-ul public de pe portul 5173/5174 și formularele de contact funcționează identic fără nicio modificare.
[ ]	**4. Configurare Hub**	Intrarea `site-admin` este `enabled: true` în `deploy/apps.config.yaml`.

[ ]	**5. Test Gateway Local**	La deschiderea `http://localhost:8080/apps/site-admin/` din Hub, panoul de admin se încarcă direct.
[ ]	**6. Test PWA Mobil**	Navigarea pe ecranul unui telefon prin Tailscale afișează corect leadurile și studiile de caz.
[ ]	**7. Backup Verificat**	Comanda `npm run backup:run` generează arhiva criptată `.age` incluzând baza WebServices.